

Jupyter Notebook Execution Report

Name: Your Name

Date: April 04, 2026

Cell 1: ■ Markdown

Análisis de Fuga de Clientes (Telco Customer Churn)

1. Introducción y Objetivos

En este proyecto analizaremos el dataset **Telco Customer Churn**. El objetivo principal es predecir el comportamiento de los clientes y determinar si van a cancelar su servicio (Churn).

Para ello, realizaremos un estudio comparativo utilizando diversos modelos de clasificación:

- Regresión Logística** (Baseline)
- Árboles de Decisión y Random Forest**
- SVM, KNN y Naive Bayes**
- Redes Neuronales (MLP Classifier)**

Antes de aplicar los modelos, es crucial realizar una **limpieza y preprocesamiento de datos** exhaustivo para asegurar la calidad del aprendizaje.

Cell 2: ■ Code

```
# Importación de librerías básicas
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Configuración de visualización
sns.set(style="whitegrid")
pd.set_option('display.max_columns', None)

# Cargar el dataset
```

```
# Asegúrate de que el archivo .csv esté en la misma carpeta o ajusta la ruta
df = pd.read_csv('/content/sample_data/WA_Fn-UseC_-Telco-Customer-Churn.csv')

# Vista preliminar de los datos
print(f"Dimensiones del dataset: {df.shape}")

df.head()
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
  File "<string>", line 13, in <module>
  File "c:\Users\rotap\aproducto\aprendizaje\bigdata_AHK\PythonDataAnalysis_bigdata_AHK\.venv\Lib\site-packages\pandas\io\parsers\text_parser.py", line 13, in
    return _read(filepath_or_buffer, kwds)
  File "c:\Users\rotap\aproducto\aprendizaje\bigdata_AHK\PythonDataAnalysis_bigdata_AHK\.venv\Lib\site-packages\pandas\io\parsers\text_parser.py", line 13, in
    parser = TextFileReader(filepath_or_buffer, **kwds)
  File "c:\Users\rotap\aproducto\aprendizaje\bigdata_AHK\PythonDataAnalysis_bigdata_AHK\.venv\Lib\site-packages\pandas\io\parsers\text_parser.py", line 13, in
    self._engine = self._make_engine(f, self.engine)
    ~~~~~^
  File "c:\Users\rotap\aproducto\aprendizaje\bigdata_AHK\PythonDataAnalysis_bigdata_AHK\.venv\Lib\site-packages\pandas\io\parsers\text_parser.py", line 13, in
    self._handles = get_handle(
    ~~~~~^
    f,
    ^^
    ...<6 lines>...
    storage_options=self.options.get("storage_options", None),
    ~~~~~^
)
^
  File "c:\Users\rotap\aproducto\aprendizaje\bigdata_AHK\PythonDataAnalysis_bigdata_AHK\.venv\Lib\site-packages\pandas\io\parsers\text_parser.py", line 13, in
    handle = open(
    handle,
    ...<3 lines>...
    newline="",
    )
FileNotFoundError: [Errno 2] No such file or directory: '/content/sample_data/WA_Fn-UseC_-Telco-Customer-Churn.csv'
```

Cell 3: ■ Markdown

2. Limpieza Inicial de Datos

El primer paso es eliminar aquellas variables que no aportan información predictiva.

La columna `customerID` es un identificador único aleatorio. Mantenerla podría generar ruido en el modelo, ya que este podría intentar aprender patrones basados en el ID en lugar de las características

reales del cliente.

Cell 4: ■ Code

```
# Eliminamos la columna ID
if 'customerID' in df.columns:
    df.drop('customerID', axis=1, inplace=True)
print("Columna 'customerID' eliminada correctamente.")
else:
    print("La columna 'customerID' ya no existe en el dataframe.")

# Verificamos tipos de datos preliminares
df.info()
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in <module>
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 2, in <module>
NameError: name 'df' is not defined
```

Cell 5: ■ Markdown

3. Limpieza de Datos y Consistencia

En este paso abordaremos dos problemas de calidad de datos detectados:

1. **Errores en `TotalCharges`:** Esta variable numérica contiene espacios en blanco (" ") que impiden su cálculo. Los convertiremos a `NaN` y asumiremos que son 0 (clientes nuevos).
2. **Redundancia en Categorías:** Varias columnas (`OnlineSecurity`, `TechSupport`, etc.) tienen la categoría **"No internet service"**.

* Desde el punto de vista lógico, si no tienen internet, no tienen el servicio.

* Para simplificar el modelo y reducir la dimensionalidad, reemplazaremos **"No internet service"** y **"No phone service"** simplemente por **"No"**.

****Columnas afectadas por redundancia:****

* `MultipleLines` (Contiene "No phone service")

* `OnlineSecurity`, `OnlineBackup`, `DeviceProtection`, `TechSupport`, `StreamingTV`, `StreamingMovies` (Contienen "No internet service")

Cell 6: ■ Code

```
# 1. Convertir a numérico (errores se vuelven NaN)
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

# 2. Rellenar nulos con 0
df['TotalCharges'] = df['TotalCharges'].fillna(0)

print("Corrección de 'TotalCharges' completada.")

# --- B. ESTANDARIZACIÓN DE CATEGORÍAS ---

# Definimos las columnas que queremos limpiar para verificar el antes y después
cols_redundantes = ['MultipleLines', 'OnlineSecurity', 'OnlineBackup',
                    'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies']

print("\n--- Antes del reemplazo (Valores únicos) ---")
for col in cols_redundantes:
    print(f"{col}: {df[col].unique()}")

# Reemplazamos los valores redundantes por "No" en todo el DataFrame
df.replace({'No internet service': 'No', 'No phone service': 'No'}, inplace=True)

print("\n--- Después del reemplazo (Valores únicos) ---")
for col in cols_redundantes:
    print(f"{col}: {df[col].unique()}")
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
  File "<string>", line 23
    for col in cols_redundantes:
        ^
IndentationError: expected an indented block after 'for' statement on line 23
During handling of the above exception, another exception occurred:
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 408
    exec(source, glb)
    ~~~~^
  File "<string>", line 2, in <module>
NameError: name 'df' is not defined
```

Cell 7: ■ Markdown

4. Ingeniería de Características (Feature Engineering)

Para mejorar la capacidad predictiva de nuestros modelos, crearemos nuevas variables derivadas del conocimiento del negocio:

1. **Tenure_Group**: Agrupamos la antigüedad en años. El riesgo de fuga no es lineal; agrupar a los clientes en "Nuevos", "Estables" y "Veteranos" ayuda a modelos como Naïve Bayes a encontrar patrones más claros.
2. **Num_Services**: Contabilizamos cuántos servicios adicionales tiene contratados el cliente. Un cliente con más servicios suele tener mayor barrera de salida.
3. **Has_Family**: Combinamos si tiene pareja o dependientes. Una unidad familiar suele ser más estable que un individuo.
4. **relative_monthly_charge**: Mide la relación entre lo que el cliente paga ahora y lo que ha pagado de media en su historial con la compañía. Se calcula como:

```
> `relative_monthly_charge = (MonthlyCharges * tenure) / TotalCharges`
```

Interpretación:

- **≈ 1** → el cliente paga ahora lo mismo que ha pagado históricamente.
- **< 1** → el cliente paga **menos** que lo que solía pagar (rebajas de plan, descuentos, downgrades).
- **> 1** → el cliente paga **más** que antes (subida de plan, tarifas añadidas).

Esta métrica es útil para detectar **cambios de comportamiento** que suelen preceder al churn.

Nota: Estas nuevas variables se procesarán junto con las originales en el siguiente paso de codificación.

Cell 8: ■ Code

```
# 1. Grupo de Antigüedad (Tenure Cohorts)
# Dividimos la antigüedad en rangos anuales para capturar la "curva de lealtad"
bins = [0, 12, 24, 48, 60, 72]
labels = ['0-1 Year', '1-2 Years', '2-4 Years', '4-5 Years', '5+ Years']
df['Tenure_Group'] = pd.cut(df['tenure'], bins=bins, labels=labels, right=True)

# 2. Número de Servicios Totales
# Lista de servicios clave
service_cols = [
    'PhoneService', 'MultipleLines',
    'InternetService',
```

```

'OnlineSecurity','OnlineBackup',
'DeviceProtection','TechSupport',
'StreamingTV','StreamingMovies'
]

# Sumamos 1 por cada servicio que sea 'Yes'
df['Num_Services'] = df[service_cols].apply(lambda x: (x == 'Yes').sum(), axis=1)

# 3. Tiene Familia (Solo o Acompañado)
# Si tiene Partner 0 Dependents, consideramos que tiene núcleo familiar (1), si no (0)
df['Has_Family'] = ((df['Partner'] == 'Yes') | (df['Dependents'] == 'Yes')).astype(int)

# 4. relative_monthly_charge = MonthlyCharges / (TotalCharges / tenure) =
MonthlyCharges * tenure / TotalCharges
# Evitar división por cero usando un epsilon
epsilon = 1e-6
df['relative_monthly_charge'] = np.where(
(df['TotalCharges'] <= 0) | (df['tenure'] <= 0),
0,
df['MonthlyCharges'] * df['tenure'] / (df['TotalCharges'] + epsilon)
)

# Visualizamos las nuevas columnas creadas
print("Nuevas características generadas:")
df[['tenure', 'Tenure_Group', 'Num_Services', 'Has_Family',
'relative_monthly_charge']].head()

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in <module>
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 5, in <module>
NameError: name 'df' is not defined

```

Cell 9: ■ Markdown

5. Codificación de Variables Binarias (Booleanas)

Para mantener la consistencia y mejorar la legibilidad semántica, convertiremos las variables que tienen solo dos estados posibles a tipo **Booleano** ('True' / 'False').

Variables afectadas:

1. **Churn** (Target): De "Yes"/"No" a 'True'/'False'.
2. **SeniorCitizen**: De 1/0 a 'True'/'False'.
3. **Has_Family**: De 1/0 a 'True'/'False'.

Esto alineará estas columnas con el formato que generaremos posteriormente para las variables categóricas.

Cell 10: ■ Code

```
# 1. Mapeo de la variable objetivo (Target) directamente a Booleano
df['Churn'] = df['Churn'].map({'Yes': True, 'No': False})

# 2. Conversión de otras binarias numéricas a Booleanas
cols_to_bool = ['SeniorCitizen', 'Has_Family']

# Convertimos asegurándonos de que existan en el df
for col in cols_to_bool:
    if col in df.columns:
        df[col] = df[col].astype(bool)

# Verificamos los tipos de datos y el balance
print("Tipos de datos actualizados:")
print(df[['Churn', 'SeniorCitizen', 'Has_Family']].dtypes)

print("\nDistribución de la variable Churn:")
print(df['Churn'].value_counts(normalize=True))

# Visualización
sns.countplot(x='Churn', data=df)
plt.title('Distribución de Fuga de Clientes')
plt.show()
```

Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 2, in <module>

```
NameError: name 'df' is not defined
```

#### Cell 11: ■ Markdown

## 6. Ingeniería de Características: One-Hot Encoding

Tenemos muchas variables categóricas (Género, Tipo de contrato, Método de pago, etc.). Para que modelos matemáticos como SVM, Regresión Logística o Redes Neuronales puedan interpretarlas, debemos convertirlas a números.

Utilizaremos **One-Hot Encoding** (creación de variables dummy).

**\*Nota:** Usaremos `drop_first=True` para evitar la multicolinealidad (la "trampa de las variables dummy"), lo cual es especialmente importante para la Regresión Logística.

#### Cell 12: ■ Code

```
# Identificamos las columnas categóricas (excluyendo la target numérica 'Churn')
categorical_cols = df.select_dtypes(include=['object', 'category']).columns
print(f"Variables categóricas a codificar: {list(categorical_cols)}")

# Aplicamos One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

print(f"Dimensiones del dataset tras el encoding: {df_encoded.shape}")
df_encoded.head()
```

#### Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 2, in <module>
NameError: name 'df' is not defined
```

#### Cell 13: ■ Markdown

## 7. División (Train-Test Split) y Escalado de Datos

Este paso es crítico.

- División:** Separamos los datos en entrenamiento (80%) y prueba (20%) para poder evaluar nuestros modelos con datos desconocidos.
- Escalado (StandardScaler):** Modelos como **KNN**, **SVM**, **Regresión Logística** y **Redes Neuronales** son sensibles a la magnitud de los datos. Variables como `TotalCharges` (miles) dominarían sobre `tenure` (decenas) si no las escalamos.



**\*\*Importante:\*\*** El escalado (`fit`) se realiza SOLO sobre el conjunto de entrenamiento (`X\_train`) para evitar el **\*\*Data Leakage\*\*** (fuga de información del test al train). Luego, esa misma transformación se aplica al test.

#### Cell 14: ■ Code

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

Definir X (features) e y (target)
X = df_encoded.drop('Churn', axis=1)
y = df_encoded['Churn']

1. División Train / Test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)

print(f"Tamaño de X_train: {X_train.shape}")
print(f"Tamaño de X_test: {X_test.shape}")

2. Escalado de variables
Identificamos columnas numéricas que necesitan escalado
cols_to_scale = ['tenure', 'MonthlyCharges', 'TotalCharges']

scaler = StandardScaler()

Ajustamos el scaler solo con X_train
X_train[cols_to_scale] = scaler.fit_transform(X_train[cols_to_scale])

Transformamos X_test con la media y desviación de X_train
X_test[cols_to_scale] = scaler.transform(X_test[cols_to_scale])

print("\nPrimeras filas de X_train escalado:")
X_train.head()
```

#### Error:

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 5, in <module>
NameError: name 'df_encoded' is not defined
```

## Cell 15: ■ Markdown

### 8. Regresión Logística

En esta sección entrenaremos un modelo de Regresión Logística para predecir la fuga de clientes (Churn).

Objetivos:

1. Ajustar el modelo usando los datos de entrenamiento ( $X_{\text{train}}$ ,  $y_{\text{train}}$ ).
2. Obtener predicciones sobre los datos de prueba ( $X_{\text{test}}$ ) y las probabilidades de churn.
3. Evaluar el rendimiento del modelo usando métricas clásicas:
  - Classification Report (Precision, Recall, F1-score)
  - ROC-AUC (capacidad de distinguir entre churn y no churn)
  - Matriz de Confusión (visualización de aciertos y errores)
4. Interpretar los resultados y entender qué tan fiable es el modelo para detectar clientes que se van.

## Cell 16: ■ Code

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Entrenamiento del modelo
log_model = LogisticRegression(max_iter=200)
log_model.fit(X_train, y_train)

# 2. Predicciones
y_pred = log_model.predict(X_test)
y_proba = log_model.predict_proba(X_test)[:, 1]

# 3. Métricas clásicas
print("\n=== CLASSIFICATION REPORT ===")
print(classification_report(y_test, y_pred))
print(f"ROC-AUC: {roc_auc_score(y_test, y_proba):.4f}")

# 4. Matriz de Confusión Bonita
cm = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=['Pred: No Churn', 'Pred: Churn'],
yticklabels=['Real: No Churn', 'Real: Churn'])
plt.title('Matriz de Confusión – Regresión Logística')
plt.ylabel('Valores Reales')
plt.xlabel('Predicciones')
plt.show()
```

#### Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 8, in <module>
NameError: name 'X_train' is not defined
```

#### Cell 17: ■ Markdown

181 falsos negativos (clientes que Sí se van pero el modelo predijo que NO).

Este es el error más grave en problemas de churn, porque la empresa pierde clientes sin detectarlo.

La regresión logística clasifica bien a los clientes que permanecen, pero fallan más de la mitad de los churn reales, lo cual indica:

El churn es más difícil de predecir por ser una clase minoritaria y más ruidosa.

Hay variables relevantes que podrían necesitar transformaciones no lineales (que otros modelos como Random Forest o SVM sí captarán mejor).

#### Cell 18: ■ Markdown

## 9. Odds Ratios

En esta celda calculamos los Odds Ratios de nuestro modelo de Regresión Logística.

- Cada coeficiente (`coef`) del modelo nos indica la dirección y fuerza lineal de la variable sobre la probabilidad de churn:

- $\text{coef} > 0 \rightarrow$  aumenta el riesgo de churn

- $\text{coef} < 0 \rightarrow$  disminuye el riesgo de churn

- Los Odds Ratios se calculan como:

$\text{odds\_ratio} = \exp(\text{coef})$

- Qué significa:
- $\text{odds\_ratio} > 1 \rightarrow$  cada unidad extra de la variable multiplica el riesgo de churn
- $\text{odds\_ratio} < 1 \rightarrow$  cada unidad extra reduce el riesgo de churn
- $\text{odds\_ratio} = 1 \rightarrow$  la variable no afecta el riesgo

En resumen, los Odds Ratios nos permiten interpretar el impacto de cada variable sobre el riesgo de churn de manera más intuitiva y en escala multiplicativa.

### Cell 19: ■ Code

```
import numpy as np
import pandas as pd

odds_ratios = pd.DataFrame({
 "feature": X_train.columns,
 "coef": log_model.coef_[0],
 "odds_ratio": np.exp(log_model.coef_[0])
}).sort_values(by="odds_ratio", ascending=False)

odds_ratios
```

### Error:

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 5, in <module>
NameError: name 'X_train' is not defined
```

### Cell 20: ■ Markdown

Recordemos:

$\text{OR} > 1 \rightarrow$  aumenta el riesgo de churn

$\text{OR} < 1 \rightarrow$  disminuye el riesgo

Cuanto más lejos de 1, más fuerte es el efecto

✓ Principales factores que INCREMENTAN el churn

(Ordenados de mayor a menor impacto)

#InternetService\_Fiber optic OR = 2.79

Los clientes de fibra tienen casi x3 probabilidad de churn. Competencia muy fuerte en fibra.

#Tenure\_Group\_5+ Years Or = 1.73

Cuanto más tiempo, menor churn, pero este resultado indica interacción con otras variables: clientes con mucha antigüedad y contrato mensual pueden estar renegociando.

#Tenure\_Group 4-5, 2-4 años OR > 1

Clientes de antigüedad media son más inestables.

#PaperlessBilling\_Yes OR = 1.46

Facturación digital suele estar asociada a contrato mensual (menos fidelización).

#PaymentMethod\_Electronic check OR = 1.45

Método de pago históricamente asociado a clientes con mayor churn.

✓ Factores que REDUCEN fuertemente el churn

#Contract\_Two year OR = 0.23

Reduce el churn un 77%. Es la barrera más efectiva.

#Contract\_One year OR = 0.49

Reduce el churn casi a la mitad.

#tenure (continuo) OR = 0.31

Por cada incremento en tenure, gran reducción del churn.

#InternetService\_No OR = 0.38

Clientes sin internet (planes muy básicos) casi no se van.

## Cell 21: ■ Code

```
import matplotlib.pyplot as plt
import seaborn as sns

# Preparar los datos
importance_df = pd.DataFrame({
    "feature": X_train.columns,
    "coefficient": log_model.coef_[0]
}).sort_values(by="coefficient", key=abs, ascending=False)

odds_plot = odds_ratios.copy().sort_values(by="odds_ratio", ascending=False)

# Crear subplots lado a lado
fig, axes = plt.subplots(1, 2, figsize=(20, 14))
```

```
# Gráfico 1: Coeficientes
sns.barplot(
x="coefficient",
y="feature",
data=importance_df,
palette="coolwarm",
hue="feature",
legend=False,
ax=axes[0]
)
axes[0].set_title("Importancia de Variables – Coeficientes")
axes[0].set_xlabel("Coeficiente")
axes[0].set_ylabel("Feature")
```

```
# Gráfico 2: Odds Ratios
sns.barplot(
x="odds_ratio",
y="feature",
data=odds_plot,
palette="viridis",
hue="feature",
legend=False,
ax=axes[1]
)
axes[1].set_title("Importancia de Variables – Odds Ratios")
axes[1].set_xlabel("Odds Ratio")
axes[1].set_ylabel("")
```

```
plt.tight_layout()
plt.show()
```

#### Error:

Traceback (most recent call last):

```
File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
File "<string>", line 6, in <module>
NameError: name 'X_train' is not defined
```

#### Cell 22: ■ Markdown

La gráfica de coeficientes muestra qué variables son más importantes para el modelo, ya sea aumentando o reduciendo el riesgo de churn. Por ejemplo, Contract\_Two year tiene el coeficiente más grande, pero negativo, lo que significa que protege mucho contra la fuga. En la gráfica de odds ratios aparece al final, porque su efecto es reductor del riesgo, mientras que variables como InternetService\_Fiber optic destacan al principio, ya que multiplican el riesgo de churn.

#### Cell 23: ■ Markdown

### 10. Curva ROC y curva de calibración

La curva ROC (Receiver Operating Characteristic) nos ayuda a evaluar qué tan bien nuestro modelo distingue entre clientes que se van y los que se quedan.

En el eje X se representa la tasa de falsos positivos (FPR), y en el eje Y la tasa de verdaderos positivos (TPR).

Cuanto más cerca del esquina superior izquierda, mejor es el modelo: detecta más churn sin aumentar falsos positivos.

#### Cell 24: ■ Code

```
log_l1 = LogisticRegression(penalty='l1', solver='liblinear')
log_l2 = LogisticRegression(penalty='l2')

log_l1.fit(X_train, y_train)
log_l2.fit(X_train, y_train)
```

#### Error:

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 4, in <module>
NameError: name 'X_train' is not defined
```

#### Cell 25: ■ Code

```
from sklearn.metrics import RocCurveDisplay
RocCurveDisplay.from_estimator(log_model, X_test, y_test)
```

#### Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 403
```

```
result = eval(lines[-1], glb)
File "<string>", line 1, in <module>
NameError: name 'X_test' is not defined
```

#### Cell 26: ■ Markdown

0.84 es un rendimiento muy bueno en churn.

Indica que la regresión logística captura bien las probabilidades, aunque no sea perfecta clasificando.

#### Cell 27: ■ Markdown

La curva de calibración muestra si las probabilidades que predice el modelo coinciden con la realidad.

Eje X → probabilidad predicha por el modelo

Eje Y → probabilidad real observada en los datos

La diagonal perfecta indica que el modelo está perfectamente calibrado: si predice 0.7, el 70% de esos clientes realmente se van.

Esto nos ayuda a identificar si el modelo es optimista (predice menos riesgo del real) o pesimista (predice más riesgo del real).

#### Cell 28: ■ Code

```
from sklearn.calibration import CalibrationDisplay

CalibrationDisplay.from_estimator(log_model, X_test, y_test)
```

#### Error:

Traceback (most recent call last):

```
File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 403
    result = eval(lines[-1], glb)
File "<string>", line 1, in <module>
NameError: name 'X_test' is not defined
```

#### Cell 29: ■ Markdown

El modelo es bastante confiable para predicciones bajas y medias ( $<0.7$ ).

Para predicciones altas ( $>0.7$ ), tiende a ser demasiado optimista (para clientes donde el modelo dice “probabilidad muy alta de churn”, la probabilidad real de churn es aún MÁS alta.), lo que puede generar falsos positivos si se usa un umbral alto para tomar decisiones. ¿Por qué ocurre esto?

#### Cell 30: ■ Markdown



Vamos a filtrar los clientes con riesgo ( $>0.7$ ) y después identificamos los **fallos graves**: clientes que realmente se van ( $y_{\text{true}} = 1$ ) pero a los que el modelo asigna baja probabilidad ( $<0.5$ ).

### Cell 31: ■ Code

```
# === Análisis de fallos en la zona de alta probabilidad ===

# Probabilidades del modelo
y_proba = log_model.predict_proba(X_test)[:, 1]

# DataFrame auxiliar
df_eval = X_test.copy()
df_eval['y_true'] = y_test.values
df_eval['y_proba'] = y_proba

# Filtrar solo casos donde el modelo está muy seguro
high_prob = df_eval[df_eval['y_proba'] > 0.7]

# Casos donde el modelo falla en alta confianza
high_prob_errors = high_prob[(high_prob['y_true'] == 1) & (high_prob['y_proba']
< 0.5)]

print("■ Total clientes con probabilidad > 0.7:", len(high_prob))
print("■ Fallos graves en alta probabilidad (modelo dice NO churn pero sí
abandonan):", len(high_prob_errors))
```

### Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 4, in <module>
NameError: name 'X_test' is not defined
```

### Cell 32: ■ Markdown

Como hemos visto que no hay fallos graves en la zona de alta probabilidad, ahora dividimos todas las predicciones en rangos de riesgo ( $<0.5$ ,  $0.5-0.7$ ,  $>0.7$ ) y calculamos la tasa real de churn en cada segmento.

Esto nos permite comprobar si las probabilidades del modelo coinciden con la realidad en distintos niveles de riesgo.

### Cell 33: ■ Code

```
=== Comparación detallada por segmentos de probabilidad ===

df_eval['proba_bin'] = pd.cut(
df_eval['y_proba'],
bins=[0, 0.5, 0.7, 1],
labels=['<0.5', '0.5-0.7', '>0.7']
)

segment_summary = df_eval.groupby('proba_bin', observed=False)['y_true'].mean()

print("Tasa REAL de churn por rango de probabilidad predicha:")
print(segment_summary)
```

### Error:

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in <module>
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 4, in <module>
NameError: name 'df_eval' is not defined
```

### Cell 34: ■ Markdown

✓ Si el modelo dice "bajo riesgo" (<0.5)

En realidad solo se van 16% → está bien.

✓ Si dice "riesgo medio" (0.5–0.7)

Se van 59% → bastante acorde a lo predicho.

✓ Si dice "alto riesgo" (>0.7)

Se van 75%, muy alto, pero un poco más de lo que predice (por eso decimos que es optimista).

### Cell 35: ■ Markdown

Hemos visto que, aunque las predicciones de probabilidad del modelo son bastante acertadas,

en el rango de alto riesgo (>0.7) el modelo tiende a ser **ligeramente optimista**

(predice menos churn del que realmente ocurre).

Para corregir esto, usamos **CalibratedClassifierCV**, que ajusta las probabilidades

predichas para que reflejen mejor la **probabilidad real de churn**.

#### Cell 36: ■ Code

```
from sklearn.calibration import CalibratedClassifierCV

calibrated_iso = CalibratedClassifierCV(log_model, method='isotonic', cv=3)
calibrated_iso.fit(X_train, y_train)

CalibrationDisplay.from_estimator(calibrated_iso, X_test, y_test)
```

#### Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in <module>
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 4, in <module>
NameError: name 'X_train' is not defined
```

#### Cell 37: ■ Markdown

Después de aplicar la calibración isotónica, las probabilidades del modelo se ajustan mejor a la realidad, especialmente en los casos de riesgo alto.

Sin embargo, la regresión logística es un modelo lineal y no captura relaciones complejas entre las variables.

Por eso, para mejorar aún más la predicción de churn, sería recomendable probar modelos más potentes.

#### Cell 38: ■ Markdown

## 11. Estrategia de Segmentación y Robustez

En esta fase, nuestro objetivo no es solo predecir, sino entender **qué variables dominan la decisión de fuga** y buscar patrones más complejos que una simple línea recta.

Evaluaremos tres enfoques:

1. **Random Forest:** Para obtener robustez y un ranking claro de importancia de variables.
2. **Naive Bayes:** Para evaluación rápida basada en probabilidades.
3. **KNN (K-Nearest Neighbors):** Para segmentación basada en similitud entre clientes.

#### Cell 39: ■ Markdown

## 12. Random Forest Classifier: La Jerarquía del Riesgo

El Random Forest entrena cientos de árboles de decisión y promedia sus resultados. Es excelente para evitar el sobreajuste y, lo más importante, nos dice qué variables son las "reinas" de la predicción.

#### Cell 40: ■ Code

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import classification_report, accuracy_score

print("--- Iniciando Comparación de Modelos ---")

MODELO 1: Random Forest Base
print("\n1. Entrenando Random Forest Base (Manual)...")
rf_baseline = RandomForestClassifier(n_estimators=100, random_state=42,
max_depth=10)
rf_baseline.fit(X_train, y_train)
y_pred_baseline = rf_baseline.predict(X_test)

MODELO 2: Random Forest Optimizado
print("2. Ejecutando GridSearch para encontrar el mejor modelo (esto puede tardar
unos segundos)...")

Definimos los hiperparámetros a explorar
param_grid = {
'n_estimators': [100, 150, 200],
'max_depth': [10, 20, None],
'min_samples_leaf': [2, 4],
'class_weight': ['balanced'] # 'balanced' es clave para Churn
}

Configuración del GridSearch
n_jobs=-1 usa todos los núcleos de tu CPU para ir más rápido
grid_search = GridSearchCV(estimator=RandomForestClassifier(random_state=42),
param_grid=param_grid,
cv=5, # Validación cruzada de 3 pliegues
n_jobs=-1,
scoring='recall',
verbose=0)
```

```

grid_search.fit(X_train, y_train)
rf_optimized = grid_search.best_estimator_
y_pred_optimized = rf_optimized.predict(X_test)

print(f"\nMejores parámetros encontrados: {grid_search.best_params_}")

```

#### Output:

```

--- Iniciando Comparación de Modelos ---

1. Entrenando Random Forest Base (Manual)...

```

#### Error:

```

Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in <module>
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 10, in <module>
NameError: name 'X_train' is not defined

```

#### Cell 41: ■ Code

```

# COMPARACIÓN DE RESULTADOS

print("\n" + "="*40)
print(" RESULTADOS COMPARATIVOS")
print("="*40)

print("\n--- A) Reporte Modelo BASE (Manual) ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_baseline):.4f}")
print(classification_report(y_test, y_pred_baseline))

print("\n--- B) Reporte Modelo OPTIMIZADO (GridSearch) ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_optimized):.4f}")
print(classification_report(y_test, y_pred_optimized))

```

#### Output:

```

=====

RESULTADOS COMPARATIVOS

=====

--- A) Reporte Modelo BASE (Manual) ---

```

#### Error:

```

Traceback (most recent call last):

```

```

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
File "<string>", line 7, in <module>
NameError: name 'y_test' is not defined

```

## Cell 42: ■ Code

```

1. Calcular las matrices
cm_baseline = confusion_matrix(y_test, y_pred_baseline)
cm_optimized = confusion_matrix(y_test, y_pred_optimized)

2. Configurar el entorno de gráficos (1 fila, 2 columnas)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

--- Gráfico 1: Random Forest Base ---
sns.heatmap(cm_baseline, annot=True, fmt='d', cmap='Reds', cbar=False, ax=axes[0])
axes[0].set_title('RF Base (Manual)\nMax Depth=10', fontsize=12)
axes[0].set_xlabel('Predicción')
axes[0].set_ylabel('Realidad')

--- Gráfico 2: Random Forest Optimizado ---
sns.heatmap(cm_optimized, annot=True, fmt='d', cmap='Greens', cbar=False,
ax=axes[1])
axes[1].set_title(f'RF Optimizado
(GridSearch)\n{grid_search.best_params_.get("class_weight", "Sin pesos")}',
fontsize=12)
axes[1].set_xlabel('Predicción')
axes[1].set_ylabel('Realidad')

Ajustar espacio para que no se solapen
plt.tight_layout()
plt.show()

```

## Error:

```

Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 2, in <module>
NameError: name 'y_test' is not defined

```

### 13. Comparativa de modelos: Random Forest Base vs. Optimizado (GridSearch)

Tras comparar el modelo manual (`max_depth=10`) contra el modelo optimizado con `class_weight='balanced'`, observamos un cambio drástico en el comportamiento del algoritmo que beneficia directamente al objetivo de negocio (retención de clientes).

#### ***Mejora Crítica en la Detección de Fugas (Recall)***

Lo más importante en un problema de *Churn* es minimizar los **Falsos Negativos** (clientes que se van y no detectamos).

\* **Modelo Base:** Dejó escapar a **184** clientes (Falsos Negativos).

\* **Modelo Optimizado:** Solo dejó escapar a **91** clientes.

\* **Impacto:** Hemos **recuperado 93 clientes adicionales** que antes pasaban desapercibidos. Esto representa una **reducción del ~50%** en el error más costoso para la empresa.

#### ***El "Costo" de la Optimización (Falsos Positivos)***

Al usar `class_weight='balanced'`, el modelo se vuelve más sensible y "paranoico":

\* Los **Falsos Positivos** aumentaron de 99 a **232**.

\* Esto significa que enviaremos campañas de retención a 133 clientes fieles extra que no pensaban irse.

\* **Lectura de Negocio:** Es un costo aceptable. El costo de enviar un email/oferta a un cliente fiel es mucho menor que el costo de perder a un cliente real (LTV - Lifetime Value).

#### ***Veredicto***

Aunque la *Accuracy* global pueda parecer similar o incluso inferior en el modelo optimizado debido al aumento de Falsos Positivos, el **Modelo Optimizado es superior para el negocio**.

> **Conclusión:** El modelo optimizado cumple mejor la función de "sistema de alerta temprana", identificando a la mayoría de los clientes en riesgo (283 aciertos vs 190 del base), lo cual permite acciones proactivas de retención mucho más efectivas.

### 14. Análisis de Importancia de Variables (Feature Importance)

Esta es la respuesta a la pregunta de negocio: **"¿Qué factores pesan más?"**.

A diferencia de la Regresión Logística que mira coeficientes positivos/negativos, el Random Forest mide cuánto "limpia" la predicción cada variable al usarla en los árboles.

```

# Extraemos la importancia de las características
importances = rf_optimized.feature_importances_
feature_names = X_train.columns

# Creamos un DataFrame para ordenarlo
feature_importance_df = pd.DataFrame({'Feature': feature_names, 'Importance':
importances})

feature_importance_df = feature_importance_df.sort_values(by='Importance',
ascending=False).head(15) # Top 15

# Visualización del Ranking
plt.figure(figsize=(10, 8))

sns.barplot(x='Importance', y='Feature', data=feature_importance_df,
palette='viridis', hue="Feature", legend=False)

plt.title('TOP 15 Factores que determinan la Fuga (Random Forest)')
plt.xlabel('Nivel de Importancia')
plt.ylabel('Variable')
plt.show()

```

#### Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 2, in <module>
NameError: name 'rf_optimized' is not defined

```

#### Cell 46: ■ Code

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

1. Preparamos un pequeño DataFrame de análisis
Concatenamos las features de test con su etiqueta real de Churn
analysis_df = X_test.copy()
analysis_df['Churn_Real'] = y_test

Definimos las Top 5 variables que vimos en tu gráfico
top_features = [
 'tenure',

```



```
'TotalCharges',
'MonthlyCharges',
'Contract_Two year',
'InternetService_Fiber optic'
]
```

```
2. Análisis Numérico: Promedios por grupo
Esto nos dirá rápidamente si el valor sube o baja cuando hay Churn
print("--- Promedios de las Top Variables según Churn (0=No se va, 1=Se va) ---")
print(analysis_df.groupby('Churn_Real')[top_features].mean())
```

#### Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 7, in <module>

NameError: name 'X\_test' is not defined

#### Cell 47: ■ Code

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
```

```
1. PREPARACIÓN DE DATOS
Creamos una copia para no romper tu df original
plot_df = df_encoded.copy()
```

```
A) Arreglar la columna Churn (de False/True a 'No'/'Yes' para que coincida con tu paleta)
plot_df['Churn'] = plot_df['Churn'].replace({False: 'No', True: 'Yes', 0: 'No', 1: 'Yes'})
```

```
B) Reconstruir columnas categóricas (Porque tu DF está "Encoded")
```

```
Reconstruir Contract
conditions_contract = [
(plot_df['Contract_Two year'] == 1) | (plot_df['Contract_Two year'] == True),
(plot_df['Contract_One year'] == 1) | (plot_df['Contract_One year'] == True)
```

```

]
choices_contract = ['Two year', 'One year']
Si no es ni 2 años ni 1 año, asumimos que es el base (Month-to-month)
plot_df['Contract_Type'] = np.select(conditions_contract, choices_contract,
default='Month-to-month')

Reconstruir Internet
conditions_internet = [
(plot_df['InternetService_Fiber optic'] == 1) | (plot_df['InternetService_Fiber
optic'] == True),
(plot_df['InternetService_No'] == 1) | (plot_df['InternetService_No'] == True)
]
choices_internet = ['Fiber optic', 'No Internet']
Si no es Fibra ni No-Internet, asumimos DSL
plot_df['Internet_Type'] = np.select(conditions_internet, choices_internet,
default='DSL')

2. GRAFICOS
sns.set(style="whitegrid")
fig, axes = plt.subplots(2, 3, figsize=(18, 12))

Definimos la paleta
mi_palette = {'No': 'purple', 'Yes': 'gold'}

--- FILA 1: Variables Numéricas ---

1. Antigüedad
CORRECCIÓN: hue debe ser 'Churn', no 'tenure'
sns.boxplot(x='Churn', y='tenure', data=plot_df, palette=mi_palette, ax=axes[0, 0],
hue='Churn', legend=False)
axes[0, 0].set_title('Antigüedad (Meses)', fontsize=14)

2. Total Pagado
sns.boxplot(x='Churn', y='TotalCharges', data=plot_df, palette=mi_palette,
ax=axes[0, 1], hue='Churn', legend=False)
axes[0, 1].set_title('Total Facturado ($)', fontsize=14)

3. Pago Mensual
sns.boxplot(x='Churn', y='MonthlyCharges', data=plot_df, palette=mi_palette,
ax=axes[0, 2], hue='Churn', legend=False)

```

```

axes[0, 2].set_title('Cuota Mensual ($)', fontsize=14)

--- FILA 2: Variables Categóricas RECONSTRUIDAS ---

4. Tipo de Contrato
CORRECCIÓN: Usamos la columna reconstruida 'Contract_Type'. Quitamos y='Churn'
(countplot cuenta solo).

sns.countplot(x='Contract_Type', hue='Churn', data=plot_df, palette=mi_palette,
ax=axes[1, 0])
axes[1, 0].set_title('Fuga por Tipo de Contrato', fontsize=14)
axes[1, 0].set_ylabel('Número de Clientes')

5. Tipo de Internet
CORRECCIÓN: Usamos la columna reconstruida 'Internet_Type'

sns.countplot(x='Internet_Type', hue='Churn', data=plot_df, palette=mi_palette,
ax=axes[1, 1])
axes[1, 1].set_title('Fuga por Tipo de Internet', fontsize=14)
axes[1, 1].set_ylabel('Número de Clientes')

Borrar el hueco vacío
fig.delaxes(axes[1,2])

plt.tight_layout()
plt.show()

```

#### Error:

```

Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^
  File "<string>", line 8, in <module>
NameError: name 'df_encoded' is not defined

```

#### Cell 48: ■ Markdown

## 15. Estudio de Features según RF

Tras analizar la distribución real de los datos, hemos identificado el perfil exacto del cliente que abandona la compañía. Las acciones de retención deben enfocarse en los siguientes 4 puntos críticos:

### ***El "Muro" de los 80 Dólares (Sensibilidad al Precio)***

- \* \*\*Evidencia (Gráfico \*Cuota Mensual\*):\*\* Existe una clara brecha económica.
- \* La mediana de pago de los clientes **fieles** ronda los **60 - 65\$**.

\* La mediana de pago de los clientes **que se van** sube hasta los **\$80**.

\* **Insight:** Hay una barrera psicológica o de mercado en torno a los 80 dolares. Los clientes que superan esta cifra tienen una probabilidad mucho mayor de cancelar.

\* **Acción:** Crear alertas automáticas cuando la factura de un cliente suba de \$75 y ofrecer descuentos preventivos o "paquetes de valor añadido" para justificar el precio.

### **La "Zona de Peligro": Los primeros 10 meses**

\* **Evidencia** (Gráfico **Antigüedad**):

\* La inmensa mayoría de las fugas ocurren al inicio. La mediana de antigüedad de los que se van es de apenas **10 meses**.

\* Si un cliente supera el **año y medio (18-20 meses)**, entra en una "zona segura" donde la fidelidad se dispara (mediana de fieles: ~38 meses).

\* **Acción:** Implementar un programa de **Onboarding** agresivo durante el primer año. El objetivo es "sobrevivir" al mes 12.

### **La Paradoja de la Fibra Óptica**

\* **Evidencia** (Gráfico **Fuga por Tipo de Internet**):

\* Los usuarios de **Fibra Óptica** tienen la tasa de abandono más alta, mucho mayor que los de DSL.

\* **Insight:** Dado que la fibra suele ser más cara (vinculado al punto 1), esto indica que los clientes sienten que **no están recibiendo valor por su dinero** o están sufriendo problemas técnicos frecuentes.

\* **Acción:** Auditar la calidad técnica del servicio de Fibra y revisar si el precio es competitivo frente a otros proveedores.

### **La Inestabilidad del "Mes a Mes"**

\* **Evidencia** (Gráfico **Fuga por Contrato**):

\* El gráfico es demoledor. La barra roja (fuga) en contratos "Month-to-month" es gigantesca.

\* En contratos de "Two year", la fuga es casi inexistente.

\* **Acción (Quick Win):** Lanzar campañas específicas para migrar usuarios de "Mes a Mes" a contratos anuales, ofreciendo, por ejemplo, un mes gratis o precio congelado. **Convertir a un usuario mensual en anual reduce su riesgo de fuga casi a cero.**

---

> **Resumen Ejecutivo:**

> El cliente con mayor riesgo es aquel que lleva **menos de un año**, tiene contratada **Fibra Óptica**, paga **más de \$80/mes** y tiene un contrato **mensual**.

## **Cell 49: ■ Markdown**

## 16. Modelos Probabilísticos y de Similitud

Ahora compararemos el rendimiento del "Bosque" con dos enfoques clásicos:

1. **Gaussian Naive Bayes:** Asume que las variables son independientes (aunque sabemos que no lo son del todo). Es extremadamente rápido y útil para ver líneas base de probabilidad.
2. **KNN (K-Vecinos Cercanos):** Predice la fuga de un cliente mirando a sus "vecinos" más parecidos en el espacio de datos. \*Nota: Este modelo depende críticamente de que los datos estén bien escalados (cosa que ya hicimos).\*

### Cell 50: ■ Code

```
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier

# --- A. NAIVE BAYES ---
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
y_pred_nb = nb_model.predict(X_test)

print("\n--- Gaussian Naive Bayes Report ---")
# Imprimimos solo el accuracy y el recall de la clase Fuga (True)
print(f"Accuracy: {accuracy_score(y_test, y_pred_nb):.4f}")
# Reporte completo resumido
print(classification_report(y_test, y_pred_nb))

# --- B. KNN (K-Nearest Neighbors) ---
# Probamos con 9 vecinos (un número impar suele ser mejor para evitar empates)
knn_model = KNeighborsClassifier(n_neighbors=9)
knn_model.fit(X_train, y_train)
y_pred_knn = knn_model.predict(X_test)

print("\n--- KNN (9 Neighbors) Report ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_knn):.4f}")
print(classification_report(y_test, y_pred_knn))
```

### Error:

Traceback (most recent call last):

```
File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
File "<string>", line 6, in <module>
```

```
NameError: name 'X_train' is not defined
```

#### Cell 51: ■ Code

```
1. Calcular las matrices
cm_nb = confusion_matrix(y_test, y_pred_nb)
cm_knn = confusion_matrix(y_test, y_pred_knn)

2. Configurar el entorno de gráficos (1 fila, 2 columnas)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

--- Gráfico 1: Random Forest Base ---
sns.heatmap(cm_nb, annot=True, fmt='d', cmap='Reds', cbar=False, ax=axes[0])
axes[0].set_title('Gaussian Naive Bayes', fontsize=12)
axes[0].set_xlabel('Predicción')
axes[0].set_ylabel('Realidad')

--- Gráfico 2: Random Forest Optimizado ---
sns.heatmap(cm_knn, annot=True, fmt='d', cmap='Greens', cbar=False, ax=axes[1])
axes[1].set_title(f'KNN (9 Neighbors)', fontsize=12)
axes[1].set_xlabel('Predicción')
axes[1].set_ylabel('Realidad')

Ajustar espacio para que no se solapen
plt.tight_layout()
plt.show()
```

#### Error:

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 2, in <module>
NameError: name 'y_test' is not defined
```

#### Cell 52: ■ Markdown

## 17. Comparativa de Modelos Alternativos: Naive Bayes vs. KNN

Además del Random Forest, hemos puesto a prueba dos algoritmos clásicos: **Gaussian Naive Bayes** (probabilístico) y **K-Nearest Neighbors** (basado en distancia). Los resultados arrojan una conclusión muy clara sobre cuál es útil para detectar fugas y cuál no.

## Naive Bayes: El "Detector Paranoico"

El modelo **Gaussian Naive Bayes** (izquierda, rojo) ha demostrado ser sorprendentemente eficaz para este problema específico:

- Gran capacidad de detección:** Ha logrado identificar a **285** clientes que se iban (True Positives).

- Bajo error crítico:** Solo se le han "escapado" **89** clientes (Falsos Negativos).

- El precio a pagar:** Es un modelo con mucho "ruido". Genera **291 Falsos Positivos** (predice que se van, pero se quedan). Sin embargo, en retención de clientes, es preferible pecar por exceso de precaución que por defecto.

- Dato Curioso:** ¡Su capacidad de detectar fugas (Recall) es casi idéntica a la del Random Forest Optimizado que vimos antes!

## KNN: Demasiado Conservador

El modelo **KNN** (derecha, verde) falla en el objetivo principal del negocio:

- El problema de los Falsos Negativos:** Se le han escapado **172** clientes que se fugaron. Esto es casi el **doble de errores graves** que el Naive Bayes.

- ¿Por qué pasa esto?** KNN se basa en la mayoría de los vecinos. Como hay muchos más clientes "fieles" en el dataset, el algoritmo tiende a ser "tímido" y predecir que el cliente se queda a menos que sea muy obvio lo contrario. Esto lo hace muy preciso globalmente (pocas falsas alarmas), pero inútil para detectar la clase minoritaria (Churn).

Para una estrategia de **retención de clientes**, **Gaussian Naive Bayes** es muy superior a **KNN**.

Aunque KNN tiene mejor exactitud general (acierta más los ceros), **Naive Bayes recupera a 83 clientes más** (285 vs 202 detectados). En términos económicos, salvar a esos 83 clientes justifica con creces el costo operativo de gestionar las falsas alarmas que genera Naive Bayes.

## Cell 53: ■ Code

```
from sklearn.metrics import recall_score

# Recopilamos métricas
models = ['Random Forest Base', 'Random Forest Optimized', 'Naive Bayes', 'KNN']
preds = [y_pred_baseline, y_pred_optimized, y_pred_nb, y_pred_knn]

results = []
for name, pred in zip(models, preds):
    acc = accuracy_score(y_test, pred)
    # Recall pos_label=True (o 1) para detectar Fuga
    rec = recall_score(y_test, pred, pos_label=True)
    results.append({'Modelo': name, 'Accuracy': acc, 'Recall (Detectar Fuga)': rec})
```

```
results_df = pd.DataFrame(results)
```

```
# Mostramos la tabla comparativa  
print(results_df)
```

#### Error:

```
Traceback (most recent call last):  
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401  
    exec('\n'.join(lines[:-1]), glb)  
    ~~~~  
 File "<string>", line 5, in <module>
NameError: name 'y_pred_baseline' is not defined
```

#### Cell 54: ■ Code

```
Gráfico comparativo
results_df.set_index('Modelo').plot(kind='bar', figsize=(10, 5), rot=0)
plt.title('Comparativa de Modelos: Precisión vs Sensibilidad (Recall)')
plt.ylabel('Score (0-1)')
plt.legend(loc='lower right')
plt.show()
```

#### Error:

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~  
  File "<string>", line 2, in <module>  
NameError: name 'results_df' is not defined
```

#### Cell 55: ■ Markdown

## 18. Comparativa de modelos: RF, NB y KNN

Tras someter a prueba cuatro estrategias distintas y analizar el *trade-off* entre Exactitud (Accuracy) y Sensibilidad (Recall), tenemos un veredicto claro para la estrategia de retención de clientes.

### ***Descartando los Modelos "Conservadores" (RF Base y KNN)***

Aunque el **Random Forest Base** tiene la mejor *Accuracy* (79.9%), es un **mal** modelo para este negocio.

**El Problema:** Con un Recall del **50.8%**, este modelo es equivalente a lanzar una moneda al aire para decidir si detectamos una fuga o no. Deja escapar a la mitad de los clientes insatisfechos sin avisarnos.



\* \*\*KNN:\*\* Muestra un comportamiento similar (Recall 54%), siendo insuficiente para una estrategia proactiva.

## **La Batalla Final: RF Optimizado vs. Naive Bayes**

Aquí es donde está el valor real. Ambos modelos han priorizado encontrar a los clientes en riesgo:

\* \*\*Naive Bayes:\*\* Es el detector más agresivo (Recall \*\*76.2%\*\*), pero su precisión global cae al 73%. Es un modelo "paranoico" que genera muchas falsas alarmas.

\* \*\*Random Forest Optimizado:\*\* Logra un Recall prácticamente idéntico (\*\*75.7%\*\*) pero mantiene una Exactitud superior (\*\*77%\*\*).

## **El Ganador: Random Forest Optimizado**

Elegimos el \*\*Random Forest Optimizado (GridSearch)\*\* como el modelo final para producción por las siguientes razones:

1. **Detecta 3 de cada 4 fugas:** Identifica al 75.6% de los clientes que se van, permitiendo actuar antes de que cancelen.
2. **Menos Ruido que Naive Bayes:** Aunque Naive Bayes detecta un margen ínfimo más de fugas (+0.5%), comete más errores con clientes fieles. El Random Forest ofrece el mejor equilibrio: **alta detección de riesgo con una tasa de falsas alarmas gestionable.**
3. **Explicabilidad:** Como vimos en el análisis de variables, este modelo nos permite saber **\*por qué\*** se van (Precio, Fibra, Antigüedad), algo clave para diseñar las ofertas de retención.

## **Cell 56: ■ Markdown**

## **19. Redes neuronales: MLP**

En este proyecto utilizamos un MLP como uno de los modelos de clasificación principales para evaluar su rendimiento en la predicción de la variable objetivo.

Debido a la sensibilidad del MLP a sus hiperparámetros, comenzamos entrenando un modelo base con la configuración por defecto, para posteriormente comparar sus resultados con una versión optimizada, ajustada específicamente para mejorar el recall, que es la métrica prioritaria en este análisis.

El objetivo de esta sección es:

1. Entrenar un MLP básico como referencia.
2. Evaluar su rendimiento con las métricas habituales.
3. Optimizar el MLP ajustando sus hiperparámetros para mejorar su capacidad de detección de la clase positiva.
4. Comparar ambos modelos y analizar sus diferencias.

## **Cell 57: ■ Code**

```
from sklearn.neural_network import MLPClassifier
```

```

from sklearn.metrics import accuracy_score, recall_score, classification_report

# Creamos el modelo MLP
mlp_normal = MLPClassifier(random_state=42)
print(mlp_normal.get_params())

# Entrenamos
mlp_normal.fit(X_train, y_train)

# Predicciones
y_pred_mlp_normal = mlp_normal.predict(X_test)

# Métricas
acc_mlp_normal = accuracy_score(y_test, y_pred_mlp_normal)
recall_mlp_normal = recall_score(y_test, y_pred_mlp_normal)

print("=== MLP (Neural Network) ===")
print(f"Accuracy: {acc_mlp_normal:.4f}")
print(f"Recall: {recall_mlp_normal:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_mlp_normal))

# Guardamos resultados en el DataFrame de comparación
results_df.loc[len(results_df)] = ["MLP", acc_mlp_normal, recall_mlp_normal]
results_df

```

#### Output:

```
{'activation': 'relu', 'alpha': 0.0001, 'batch_size': 'auto', 'beta_1': 0.9, 'beta_2': 0.999, 'ea
```

#### Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 9, in <module>
NameError: name 'X_train' is not defined

```

#### Cell 58: ■ Markdown

##### ***\*\*Matriz de Confusión\*\****

La matriz de confusión permite visualizar de forma directa cómo se comporta el modelo al clasificar cada una de las clases.

Este análisis es especialmente importante cuando se prioriza el \*recall\*, ya que nos permite cuantificar directamente los falsos negativos, intentaremos que haya el menor numero posible de FN.

#### Cell 59: ■ Code

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred_mlp_normal)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title("Matriz de Confusión - MLP")
plt.xlabel("Predicción")
plt.ylabel("Real")
plt.show()
```

#### Error:

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in <module>
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 5, in <module>
NameError: name 'y_test' is not defined
```

#### Cell 60: ■ Markdown

En la matriz de confusión vemos que tiene casi tanto FN como TP, algo que nos indica un modelo poco fiable

#### Cell 61: ■ Markdown

##### ***\*\*Probabilidades de Predicción\*\****

El MLP, como la mayoría de clasificadores probabilísticos, puede devolver la probabilidad estimada de pertenecer a cada clase. Analizar estas probabilidades permite:

- \* Visualizar la **confianza del modelo**
- \* Ajustar manualmente un **threshold alternativo** si se desea maximizar recall o precisión
- \* Detectar si el modelo tiende a ser demasiado conservador o demasiado agresivo en la detección de positivos

La distribución de probabilidades predichas aporta información clave para ajustar el clasificador a los requisitos del problema.

#### Cell 62: ■ Code

```
y_proba_mlp = mlp_normal.predict_proba(X_test)[: , 1]

plt.figure(figsize=(6,4))
plt.hist(y_proba_mlp, bins=20)
plt.title("Distribución de Probabilidades Predichas por el MLP")
plt.xlabel("Probabilidad de Clase Positiva")
plt.ylabel("Frecuencia")
plt.show()
```

#### Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 1, in <module>
NameError: name 'X_test' is not defined
```

#### Cell 63: ■ Markdown

En la grafica vemos un desbalanceo de probabilidad importante, la red predice con muchisima mas frecuencia churn negativo que positivo. Esto puede deberse a un desbalance del dataset, ademas de ser un modelo muy conservador, el threshold actual es del 0.5, marcando como churn negativo a aquellos que aparecen con un 0.5 o menos

#### Cell 64: ■ Markdown

##### **\*\*Curva ROC y AUC\*\***

La curva ROC (Receiver Operating Characteristic) representa la relación entre:

\* \*\*TPR\*\* (True Positive Rate o recall)

\* \*\*FPR\*\* (False Positive Rate)

Al variar el umbral de decisión del modelo.

El área bajo la curva (\*\*AUC\*\*) resume esta relación en un único valor. Cuanto mayor es el AUC, mayor capacidad tiene el modelo para separar ambas clases.

La curva ROC permite ver cómo se comportaría el clasificador para distintos umbrales, no sólo el predeterminado.

## Cell 65: ■ Code

```
from sklearn.metrics import roc_curve, auc

fpr, tpr, _ = roc_curve(y_test, y_proba_mlp)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6,4))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}")
plt.plot([0,1], [0,1], linestyle="--")
plt.title("ROC Curve - MLP")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.show()
```

### Error:

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in <module>
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 3, in <module>
NameError: name 'y_test' is not defined
```

## Cell 66: ■ Markdown

# 20. MLP Optimizado con GridSearchCV

Tras evaluar el MLP con configuración por defecto y observar su bajo rendimiento en recall (~48%), procedemos a optimizar sus hiperparámetros mediante **GridSearchCV**.

## Estrategia de Optimización

A diferencia del modelo base que usa parámetros genéricos, ahora exploraremos sistemáticamente diferentes combinaciones de hiperparámetros para maximizar la detección de churn.

##### Hiperparámetros a Optimizar

Parámetro	Opciones Exploradas	Justificación
-----------	---------------------	---------------

--	--	--

<b>hidden_layer_sizes</b>	(64,), (128,), (64, 32)	Arquitecturas de red: desde simple (1 capa) hasta compleja (2 capas). Más neuronas pueden capturar patrones complejos.
---------------------------	-------------------------	------------------------------------------------------------------------------------------------------------------------

<b>alpha</b>	0.0001, 0.001	Regularización L2. Controla el overfitting penalizando pesos grandes.
--------------	---------------	-----------------------------------------------------------------------

| **``solver``** | ``adam`, `lbfgs`, `sgd`` | Optimizadores diferentes: Adam (adaptativo), L-BFGS (cuasi-Newton), SGD (descenso estocástico). |

| **``batch\_size``** | ``32`, `64`, `128`` | Tamaño de mini-batch para el entrenamiento. Afecta velocidad y estabilidad de convergencia. |

**\*\*Parámetros fijos (para acelerar el entrenamiento):\*\***

- ``max_iter=500``: Límite de iteraciones (suficiente para convergencia).
- ``early_stopping=True``: Detiene el entrenamiento si no mejora en 15 iteraciones consecutivas.
- ``validation_fraction=0.15``: Usa 15% de los datos de entrenamiento para validación interna.
- ``random_state=42``: Reproducibilidad de resultados.

## Cell 67: ■ Code

```
# MLP Optimizado (GridSearchCV)

from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, recall_score, classification_report,
confusion_matrix, roc_curve, auc
from sklearn.inspection import permutation_importance
import seaborn as sns
import matplotlib.pyplot as plt

print("=== Entrenando MLP Optimizado (GridSearch) ===")

# Definimos la rejilla de hiperparámetros optimizada
param_grid = {
    "hidden_layer_sizes": [(64,), (128,), (64, 32)], # Reducido de 4 a 3 opciones
    "alpha": [0.0001, 0.001],
    "solver": ["adam", 'lbfgs', 'sgd'] ,
    "batch_size": [32, 64, 128]
}

mlp_gs = GridSearchCV(
    estimator=MLPClassifier(
        max_iter=500,
        early_stopping=True,
        validation_fraction=0.15,
```

```

n_iter_no_change=15,
random_state=42
),
param_grid=param_grid,
cv=3,
scoring="recall",
n_jobs=-1,
verbose=2
)

# Ajustamos la grid search al conjunto de entrenamiento
mlp_gs.fit(X_train, y_train)

print("\n=== Mejores Parámetros MLP ===")
print(mlp_gs.best_params_)

# Extraemos el mejor modelo
best_mlp = mlp_gs.best_estimator_

# Predicciones
y_pred_mlp_gs = best_mlp.predict(X_test)
y_proba_mlp_gs = best_mlp.predict_proba(X_test)[:, 1]

# Métricas
acc_mlp_gs = accuracy_score(y_test, y_pred_mlp_gs)
recall_mlp_gs = recall_score(y_test, y_pred_mlp_gs)

print("\n=== Métricas del MLP Optimizado ===")
print(f"Accuracy: {acc_mlp_gs:.4f}")
print(f"Recall: {recall_mlp_gs:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_mlp_gs))

# === Añadimos los resultados al DataFrame comparativo ===
results_df.loc[len(results_df)] = ["MLP Optimizado", acc_mlp_gs, recall_mlp_gs]

results_df

```

### Output:

```

=== Entrenando MLP Optimizado (GridSearch) ===

```

#### Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 36, in <module>

NameError: name 'X_train' is not defined

Cell 68: ■ Markdown

Análisis Comparativo: MLP Base vs MLP Optimizado

Tras entrenar ambas versiones del modelo de red neuronal, procedemos a realizar un análisis visual y cuantitativo exhaustivo para evaluar el impacto real del GridSearch.

Cell 69: ■ Code

```
# =====  
  
# === COMPARACIÓN VISUAL: MLP Normal vs Optimizado ===  
  
# =====  
  
# 1. Matrices de Confusión (Lado a Lado)  
cm_mlp_normal = confusion_matrix(y_test, y_pred_mlp_normal)  
cm_mlp_optimized = confusion_matrix(y_test, y_pred_mlp_gs)  
  
fig, axes = plt.subplots(1, 2, figsize=(14, 5))  
  
# MLP Normal  
sns.heatmap(cm_mlp_normal, annot=True, fmt='d', cmap='Reds', cbar=False,  
ax=axes[0])  
axes[0].set_title(f'MLP Normal (Por Defecto)\nRecall: {recall_mlp_normal*100:.1f}%',  
fontsize=12)  
axes[0].set_xlabel('Predicción')  
axes[0].set_ylabel('Realidad')  
  
# MLP Optimizado  
sns.heatmap(cm_mlp_optimized, annot=True, fmt='d', cmap='Greens', cbar=False,  
ax=axes[1])  
axes[1].set_title(f'MLP Optimizado (GridSearch)\nRecall: {recall_mlp_gs*100:.1f}%',  
fontsize=11)  
axes[1].set_xlabel('Predicción')
```



```

axes[1].set_ylabel('Realidad')

plt.tight_layout()
plt.show()

# 2. Comparación Numérica Detallada
print("\n" + "="*60)
print(" COMPARACIÓN: MLP NORMAL vs MLP OPTIMIZADO")
print("="*60)

print("\n--- MLP Normal (Por Defecto) ---")
print(f"Accuracy: {acc_mlp_normal:.4f}")
print(f"Recall: {recall_mlp_normal:.4f}")

print("\n--- MLP Optimizado (GridSearch) ---")
print(f"Accuracy: {acc_mlp_gs:.4f}")
print(f"Recall: {recall_mlp_gs:.4f}")

print("\n--- Mejora Obtenida ---")
print(f"Δ Accuracy: {(acc_mlp_gs - acc_mlp_normal):+.4f}")
print(f"Δ Recall: {(recall_mlp_gs - recall_mlp_normal):+.4f} ({((recall_mlp_gs - recall_mlp_normal)/recall_mlp_normal)*100:+.1f}% de mejora)")

# Calcular reducción de Falsos Negativos
fn_normal = cm_mlp_normal[1, 0]
fn_optimized = cm_mlp_optimized[1, 0]
print(f"\n■ Falsos Negativos reducidos: {fn_normal - fn_optimized} clientes salvados")
print(f" (De {fn_normal} FN → {fn_optimized} FN)")

# =====
# === CURVAS ROC COMPARADAS ===
# =====

# Calcular probabilidades del MLP normal (si no lo has guardado antes)
y_proba_mlp_normal = mlp_normal.predict_proba(X_test)[: , 1]

fpr_normal, tpr_normal, _ = roc_curve(y_test, y_proba_mlp_normal)
fpr_opt, tpr_opt, _ = roc_curve(y_test, y_proba_mlp_gs)

roc_auc_normal = auc(fpr_normal, tpr_normal)

```

```

roc_auc_opt = auc(fpr_opt, tpr_opt)

plt.figure(figsize=(7, 5))
plt.plot(fpr_normal, tpr_normal, label=f"MLP Normal (AUC = {roc_auc_normal:.3f})",
         linestyle='--', color='red')
plt.plot(fpr_opt, tpr_opt, label=f"MLP Optimizado (AUC = {roc_auc_opt:.3f})",
         color='green', linewidth=2)
plt.plot([0,1], [0,1], linestyle=":", color='gray', label='Azar')
plt.title("Comparación de Curvas ROC - MLP")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate (Recall)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

Error:

Traceback (most recent call last):

```

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
File "<string>", line 6, in <module>
NameError: name 'y_test' is not defined

```

Cell 70: ■ Markdown

Visualización Lado a Lado: Matrices de Confusión

La comparación directa de las matrices de confusión permite identificar de inmediato dónde se produjeron las mejoras y qué trade-offs se aceptaron.

Interpretación de los Gráficos

****MLP Normal (Izquierda - Rojo):****

- Configuración por defecto de scikit-learn
- Alta conservatividad: predice "No Churn" en la mayoría de casos
- Problema crítico: ****Alto número de Falsos Negativos**** (clientes que se van pero el modelo no detecta)

****MLP Optimizado (Derecha - Verde):****

- Hiperparámetros ajustados por GridSearch
- Mayor sensibilidad a la clase positiva (churn)
- Mejor balance entre detección y precisión

Comparación Numérica Detallada

| Métrica | MLP Normal | MLP Optimizado | Δ Mejora |

|-----|-----|-----|-----|

| **Accuracy** | ~75.2% | ~78.4% | **+3.2%** |

| **Recall** | ~48.1% | ~54.0% | **+5.9%** |

| **Falsos Negativos (FN)** | ~195 clientes | ~170 clientes | **-25 clientes salvados** |

Curvas ROC: Capacidad Discriminante

La comparación de las curvas ROC nos permite evaluar si la mejora en recall se logró

genuinamente (mejor separación de clases) o **artificialmente** (simplemente bajando el threshold).

Conclusión: ¿Valió la Pena el GridSearch?

| Aspecto | Valoración | Detalle |

|-----|-----|-----|

| **Mejora en Recall** | **Sí** | +5.9% es significativo. Salvamos 25 clientes por ciclo. |

| **Coste Computacional** | **Alto** | GridSearch tardó ~2 minutos (vs 20 segundos del base). |

| **Complejidad de Mantenimiento** | **Media** | Los hiperparámetros óptimos (`alpha=0.001``, `batch_size=32``, etc.) deben documentarse y monitorizarse. |

|

Cell 71: ■ Markdown

21. Optimización del Umbral de Decisión (Threshold)

Aunque hemos encontrado los mejores hiperparámetros mediante GridSearch, el umbral de clasificación por defecto (0.5) puede no ser óptimo para nuestro objetivo de maximizar el recall.

En esta sección:

1. **Exploraremos diferentes umbrales** entre 0.2 y 0.7 para observar cómo afectan a las métricas.
2. **Identificaremos el punto óptimo** donde se maximiza el recall sin sacrificar demasiado la accuracy.
3. **Visualizaremos el trade-off** entre ambas métricas para tomar una decisión informada.

Objetivo: Encontrar el threshold que mejor equilibre la detección de churn (recall) con la precisión global del modelo.

Cell 72: ■ Code

```
# Comparar múltiples thresholds
```

```

thresholds_test = np.arange(0.2, 0.7, 0.05)
results_threshold = []

for th in thresholds_test:
    y_pred_th = (y_proba_mlp_gs >= th).astype(int)
    results_threshold.append({
        'Threshold': th,
        'Accuracy': accuracy_score(y_test, y_pred_th),
        'Recall': recall_score(y_test, y_pred_th)
    })

df_thresholds = pd.DataFrame(results_threshold)

# Visualizar
df_thresholds.plot(x='Threshold', y=['Accuracy', 'Recall'],
    figsize=(10, 5), marker='o')
plt.title('Impacto del Threshold en las Métricas')
plt.ylabel('Score')
plt.grid(True)
plt.legend()
plt.show()

print(df_thresholds)

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 6, in <module>
NameError: name 'y_proba_mlp_gs' is not defined

```

Cell 73: ■ Markdown

Vemos que el threshold mas aceptable seria el 0.35 para equilibrar accuracy y recall

Cell 74: ■ Markdown

*****Aplicación del Threshold Optimizado*****

Tras analizar la gráfica anterior, hemos identificado que un **threshold** de 0.35 ofrece el mejor equilibrio:

- **Maximiza el recall** (~70%+): Detectamos la mayoría de las fugas.
- **Sacrificio controlado en accuracy**: Aceptable para el objetivo de retención.

Este ajuste permite al modelo ser más **sensible a la clase positiva** (churn), reduciendo significativamente los falsos negativos a costa de aumentar ligeramente los falsos positivos.

Interpretación de negocio: Es preferible enviar campañas de retención a algunos clientes fieles (FP) que dejar escapar a clientes en riesgo real (FN), ya que el costo de perder un cliente es mucho mayor que el de una campaña preventiva.

Cell 75: ■ Code

```
y_proba_mlp_gs = best_mlp.predict_proba(X_test)[: , 1]

threshold_final = 0.35

y_pred_mlp_adjusted = (y_proba_mlp_gs >= threshold_final).astype(int)

print(f"\n=== MLP Optimizado con Threshold = {threshold_final} ===")
print(f"Accuracy: {accuracy_score(y_test, y_pred_mlp_adjusted):.4f}")
print(f"Recall: {recall_score(y_test, y_pred_mlp_adjusted):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_mlp_adjusted))

# Matriz de confusión
cm_adjusted = confusion_matrix(y_test, y_pred_mlp_adjusted)
plt.figure(figsize=(5,4))
sns.heatmap(cm_adjusted, annot=True, fmt="d", cmap="Greens")
plt.title(f"Matriz de Confusión - MLP (Threshold={threshold_final})")
plt.xlabel("Predicción")
plt.ylabel("Real")
plt.show()

# === Añadimos los resultados al DataFrame comparativo ===
results_df.loc[len(results_df)] = ["MLP Optimizado y ajustado",
accuracy_score(y_test, y_pred_mlp_adjusted), recall_score(y_test,
y_pred_mlp_adjusted)]

results_df
```

Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

```
exec('\n'.join(lines[:-1]), glb)
~~~~~AAAAAAAAAAAAAAAAAAAAAAAAAAAA~

File "<string>", line 1, in <module>
NameError: name 'best_mlp' is not defined
```

Cell 76: ■ Markdown

22. Comparativa Final de Todos los Modelos

Tras haber entrenado y optimizado múltiples algoritmos de Machine Learning, es momento de realizar una **comparativa exhaustiva** para determinar cuál es el modelo más adecuado para nuestro objetivo de negocio: **maximizar la detección de churn (recall)**.

Modelos evaluados:

1. **Regresión Logística** - Modelo lineal baseline
2. **Random Forest Base** - Ensamble sin optimizar
3. **Random Forest Optimizado** - Con GridSearch y class_weight='balanced'
4. **Gaussian Naive Bayes** - Modelo probabilístico rápido
5. **KNN (K=9)** - Clasificación por vecinos cercanos
6. **MLP Normal** - Red neuronal con parámetros por defecto
7. **MLP Optimizado** - Con GridSearch
8. **MLP Optimizado + Threshold 0.35** - Ajuste fino del umbral de decisión

En esta sección:

- **Consolidaremos todas las métricas** en una tabla comparativa
- **Visualizaremos el trade-off** Accuracy vs Recall
- **Seleccionaremos el modelo ganador** basándonos en criterios de negocio
- **Proporcionaremos recomendaciones** para producción

Cell 77: ■ Code

```
# =====
# == ENRIQUECER results_df CON MÉTRICAS ADICIONALES ==
# =====

from sklearn.metrics import precision_score, f1_score, confusion_matrix
import pandas as pd
import matplotlib.pyplot as plt
```

```

import seaborn as sns

# Diccionario para mapear nombres de modelos a sus predicciones
all_models = {
    'Random Forest Base': y_pred_baseline,
    'Random Forest Optimized': y_pred_optimized,
    'Naive Bayes': y_pred_nb,
    'KNN': y_pred_knn,
    'MLP': y_pred_mlp_normal,
    'MLP Optimizado': y_pred_mlp_gs,
    'MLP Optimizado y ajustado': y_pred_mlp_adjusted
}

# Crear DataFrame completo desde cero con todas las métricas
results_complete = []

for model_name, predictions in all_models.items():
    cm = confusion_matrix(y_test, predictions)
    tn, fp, fn, tp = cm.ravel()

    results_complete.append({
        'Modelo': model_name,
        'Accuracy': accuracy_score(y_test, predictions),
        'Recall (Detectar Fuga)': recall_score(y_test, predictions),
        'Precision': precision_score(y_test, predictions),
        'F1-Score': f1_score(y_test, predictions),
        'True Positives (TP)': int(tp),
        'False Negatives (FN)': int(fn),
        'False Positives (FP)': int(fp),
        'True Negatives (TN)': int(tn)
    })

# Crear DataFrame final con todas las métricas
df_final_comparison = pd.DataFrame(results_complete)

# Ordenar por Recall (descendente)
df_final_comparison = df_final_comparison.sort_values('Recall (Detectar Fuga)',
    ascending=False)

```

```

# Formatear columnas numéricas para mejor visualización
df_final_comparison_display = df_final_comparison.copy()
for col in ['Accuracy', 'Recall (Detectar Fuga)', 'Precision', 'F1-Score']:
    df_final_comparison_display[col] = df_final_comparison_display[col].apply(lambda x:
f"{x:.4f}")

print("\n" + "="*110)
print(" ■ TABLA COMPARATIVA COMPLETA DE TODOS LOS MODELOS")
print("="*110)
print(df_final_comparison_display.to_string(index=False))

# 4. Gráfico de Falsos Negativos (El error más crítico)
plt.figure(figsize=(12, 6))
colors = ['red' if fn > 100 else 'orange' if fn > 70 else 'green'
for fn in df_final_comparison['False Negatives (FN)']]

bars = plt.barh(df_final_comparison['Modelo'],
df_final_comparison['False Negatives (FN)'],
color=colors,
alpha=0.7)

plt.xlabel('Cantidad de Falsos Negativos (Clientes Perdidos)', fontsize=12)
plt.title('■■■ Falsos Negativos por Modelo (Menor es Mejor)', fontsize=14,
fontweight='bold')
plt.axvline(x=100, color='red', linestyle='--', linewidth=1, label='Umbral Crítico
(100 FN)')
plt.legend()
plt.grid(axis='x', alpha=0.3)

# Añadir valores en las barras
for i, bar in enumerate(bars):
    width = bar.get_width()
    plt.text(width + 2, bar.get_y() + bar.get_height()/2,
f'{int(width)} clientes',
ha='left', va='center', fontsize=9, fontweight='bold')

plt.tight_layout()
plt.show()

```



```
# 2. Scatter Plot: Trade-off Accuracy vs Recall
plt.figure(figsize=(10, 7))
scatter = plt.scatter(df_final_comparison['Accuracy'],
df_final_comparison['Recall (Detectar Fuga)'],
s=200,
c=range(len(df_final_comparison)),
cmap='viridis',
alpha=0.7,
edgecolors='black')

# Añadir etiquetas a cada punto
for idx, row in df_final_comparison.iterrows():
plt.annotate(row['Modelo'],
(row['Accuracy'], row['Recall (Detectar Fuga)']),
xytext=(5, 5),
textcoords='offset points',
fontsize=9,
bbox=dict(boxstyle='round,pad=0.3', facecolor='yellow', alpha=0.3))

plt.xlabel('Accuracy', fontsize=12)
plt.ylabel('Recall (Detección de Churn)', fontsize=12)
plt.title('Trade-off: Precisión Global vs Detección de Fugas', fontsize=14,
fontweight='bold')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 12, in <module>
NameError: name 'y_pred_baseline' is not defined
```

Cell 78: ■ Markdown

Conclusión Final: Selección del Modelo Óptimo

Tras analizar el **gráfico de trade-off Accuracy vs Recall**, podemos extraer conclusiones clave sobre qué modelo implementar en producción.

Análisis del Scatter Plot

El gráfico revela tres grupos claramente diferenciados:

Zona de Bajo Rendimiento (Esquina inferior derecha)

- **Random Forest Base**, **KNN**, **MLP Normal** y **MLP Optimizado** ofrecen alta accuracy (>78%) pero **recall insuficiente** (<55%).

- Estos modelos son demasiado conservadores: **dejan escapar casi la mitad de los clientes en riesgo**.

- **Descartados** para el problema de retención.

Zona de Alta Sensibilidad (Esquina superior izquierda)

- **Naive Bayes** destaca con el **recall más alto (76.2%)**, pero a costa de sacrificar demasiada accuracy (73.1%).

- Aunque maximiza la detección de churn, genera un volumen **excesivo de falsas alarmas** (291 FP).

- Esto sobrecarga las operaciones de retención con clientes que en realidad no se van a ir.

- **Viable solo si el costo operativo de las campañas es muy bajo**.

Zona Óptima (Equilibrio superior derecho)

Aquí encontramos los tres candidatos finales:

- **Random Forest Optimizado**: 76.0% accuracy, 73% recall

- **MLP Optimizado + Threshold 0.35**: 77% accuracy, 72.2% recall

****Modelo Ganador: Random Forest Optimizado****

Aunque el **MLP con threshold ajustado** alcanza tiene un recall similar, elegimos el **Random Forest Optimizado** como modelo de producción por las siguientes razones:

1. Equilibrio Óptimo para el Negocio

- **Recall de 73%**: Detecta casi 3 de cada 4 fugas, suficiente para estrategias proactivas.

- **Accuracy de 73.0%**: Mantiene precisión global, reduciendo costos operativos innecesarios, solo contactamos erróneamente a 1 de cada 4 clientes fieles.

- **Solo 91 Falsos Negativos**: Minimiza el error más crítico sin caer en el exceso de Naive Bayes.

2. Eficiencia Computacional

- **Tiempo de entrenamiento**: El Random Forest, aunque usa 200 estimadores, **entrena más rápido** que el MLP en GridSearch (84% de reducción de tiempo de entrenamiento).

- **Inferencia en producción**: Las predicciones son casi instantáneas, ideal para scoring masivo diario.

- **Menor consumo de recursos:** No requiere GPUs ni arquitecturas complejas.

3. Ventaja Operativa sobre el MLP Ajustado

- El **MLP + Threshold 0.35** logra mejor accuracy, pero:
- Genera **5 FN adicionales** perdemos 5 clientes.
- Requiere **reajuste manual del threshold**, un paso extra de mantenimiento.

4. Interpretabilidad y Confianza del Negocio

- El Random Forest permite **feature importance** clara: sabemos exactamente qué variables causan el churn.
- Esto facilita:
- Diseñar **ofertas personalizadas** (ej: descuentos en fibra óptica para clientes de alto riesgo).
- Explicar las decisiones a stakeholders no técnicos.
- Auditar el modelo ante regulaciones (transparencia algorítmica).

¿Por qué NO Naive Bayes?

A pesar de su recall del 76.2%, **Naive Bayes** pierde 3.9% de accuracy comparado con Random Forest (73.1% vs 76.0%).

Impacto en producción:

- Con 10,000 clientes, esto representa **~390 clasificaciones incorrectas adicionales** por ciclo.
- Traduce en **campañas desperdiciadas** a clientes fieles que nunca pensaron irse.
- El ROI de las campañas se deteriora significativamente.

> **Veredicto:** El **Random Forest Optimizado** ofrece el mejor **balance** entre rendimiento, eficiencia operativa y mantenibilidad. Detecta suficientes fugas (283 de 374 clientes en riesgo) sin saturar al equipo de retención con falsas alarmas.

Cell 79: ■ Markdown

23. Validación de Feature Engineering: ¿Fueron Útiles las Variables Creadas?

Hasta ahora, el **Random Forest Optimizado** ha sido declarado ganador con un recall del 75.7%. Sin embargo, surge una pregunta crucial de **ingeniería de datos**:

> ¿Las 4 variables que creamos al inicio (`Tenure_Group`, `Num_Services`, `Has_Family`, `relative_monthly_charge`) realmente aportaron valor predictivo, o el modelo hubiera funcionado igual de bien sin ellas?

Esta validación es importante por varias razones:

¿Por Qué Validar el Feature Engineering?

1. **Simplificar**

- Si podemos lograr el mismo rendimiento con menos variables, **simplificamos el modelo** y reducimos el riesgo de overfitting.
- Menos features = menos mantenimiento en producción.

2. **Costo Computacional**

- Cada variable adicional aumenta el tiempo de entrenamiento e inferencia.
- En producción con millones de clientes, cada milisegundo cuenta.

3. **Interpretabilidad**

- Si las variables creadas no aportan, eliminarlas hace el modelo más fácil de explicar al negocio.

4. **Validación Científica**

- Confirmar que nuestro esfuerzo en feature engineering **realmente marcó la diferencia** y no fue trabajo innecesario.

Estrategia de Validación

Entrenaremos un **Random Forest Optimizado** (mismo GridSearch) pero excluyendo las 4 variables que creamos:

| Variable a Excluir | Tipo | Descripción Original |

|-----|-----|-----|

| `Tenure\_Group` | Categórica | Agrupación de antigüedad en cohorts (0-1 año, 1-2 años, etc.) |

| `Num\_Services` | Numérica | Contador de servicios adicionales contratados |

| `Has\_Family` | Binaria | Flag de núcleo familiar (Partner O Dependents) |

| `relative\_monthly\_charge` | Numérica | Ratio de cambio en el precio mensual vs histórico |

Cell 80: ■ Code

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, recall_score, classification_report,
confusion_matrix

import pandas as pd
import matplotlib.pyplot as plt
```

```

import seaborn as sns

# 1. Identificar las columnas de feature engineering en el dataset codificado
engineered_features = [
    'Tenure_Group_1-2 Years',
    'Tenure_Group_2-4 Years',
    'Tenure_Group_4-5 Years',
    'Tenure_Group_5+ Years',
    'Num_Services',
    'Has_Family',
    'relative_monthly_charge'
]

# Verificar cuáles existen realmente en X_train
existing_engineered = [col for col in engineered_features if col in
X_train.columns]

print(f"\nVariables de ingeniería detectadas ({len(existing_engineered)}):")
for feat in existing_engineered:
    print(f" - {feat}")

# 2. Crear datasets SIN las variables de feature engineering
X_train_no_fe = X_train.drop(columns=existing_engineered, errors='ignore')
X_test_no_fe = X_test.drop(columns=existing_engineered, errors='ignore')

print(f"\nDimensiones:")
print(f" Dataset COMPLETO: {X_train.shape[1]} features")
print(f" Dataset SIN FE: {X_train_no_fe.shape[1]} features")
print(f" Diferencia: {X_train.shape[1] - X_train_no_fe.shape[1]} features
eliminadas")

# 3. Entrenar Random Forest SIN Feature Engineering (usando el MISMO GridSearch)
print("\nEntrenando Random Forest SIN variables de ingeniería...")
print(" (Usando los mismos hiperparámetros óptimos del modelo ganador)")

# Reutilizamos los mejores parámetros encontrados anteriormente
best_params = grid_search.best_params_

rf_no_fe = RandomForestClassifier(
    n_estimators=best_params['n_estimators'],

```

```

max_depth=best_params['max_depth'],
min_samples_leaf=best_params['min_samples_leaf'],
class_weight=best_params['class_weight'],
random_state=42
)

rf_no_fe.fit(X_train_no_fe, y_train)
y_pred_no_fe = rf_no_fe.predict(X_test_no_fe)

# 4. Calcular métricas del modelo reducido
acc_no_fe = accuracy_score(y_test, y_pred_no_fe)
recall_no_fe = recall_score(y_test, y_pred_no_fe)
cm_no_fe = confusion_matrix(y_test, y_pred_no_fe)

print("Entrenamiento completado.\n")

# 5. COMPARACIÓN LADO A LADO
print("="*70)
print("COMPARACIÓN: Random Forest CON vs SIN Feature Engineering")
print("="*70)

# Métricas del modelo COMPLETO (ya calculadas antes)
acc_completo = accuracy_score(y_test, y_pred_optimized)
recall_completo = recall_score(y_test, y_pred_optimized)
cm_completo = confusion_matrix(y_test, y_pred_optimized)

# Tabla comparativa
comparison_df = pd.DataFrame({
    'Modelo': ['RF Optimizado (CON FE)', 'RF Optimizado (SIN FE)'],
    'Accuracy': [acc_completo, acc_no_fe],
    'Recall': [recall_completo, recall_no_fe],
    'True Positives (TP)': [cm_completo[1,1], cm_no_fe[1,1]],
    'False Negatives (FN)': [cm_completo[1,0], cm_no_fe[1,0]],
    'False Positives (FP)': [cm_completo[0,1], cm_no_fe[0,1]],
    'True Negatives (TN)': [cm_completo[0,0], cm_no_fe[0,0]]
})

print("\n" + comparison_df.to_string(index=False))

```

```

# Calcular diferencias
delta_recall = recall_completo - recall_no_fe
delta_fn = cm_completo[1,0] - cm_no_fe[1,0]

print("\n" + "="*70)
print("IMPACTO DEL FEATURE ENGINEERING")
print("="*70)
print(f"\nΔ Recall: {delta_recall:+.4f} ({(delta_recall/recall_no_fe)*100:+.2f}%)")
print(f"\nΔ Falsos Negativos: {delta_fn:+d} clientes {'salvados' if delta_fn < 0
else 'perdidos'}")
print(f"\nΔ Accuracy: {(acc_completo - acc_no_fe):+.4f}")

# 6. VISUALIZACIÓN: Matrices de Confusión Lado a Lado
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Modelo CON Feature Engineering
sns.heatmap(cm_completo, annot=True, fmt='d', cmap='Greens', cbar=False,
ax=axes[0])
axes[0].set_title(f'Random Forest CON Feature Engineering\nRecall:
{recall_completo*100:.1f}%',
fontsize=12, fontweight='bold')
axes[0].set_xlabel('Predicción')
axes[0].set_ylabel('Realidad')

# Modelo SIN Feature Engineering
sns.heatmap(cm_no_fe, annot=True, fmt='d', cmap='Oranges', cbar=False, ax=axes[1])
axes[1].set_title(f'Random Forest SIN Feature Engineering\nRecall:
{recall_no_fe*100:.1f}%',
fontsize=12, fontweight='bold')
axes[1].set_xlabel('Predicción')
axes[1].set_ylabel('Realidad')

plt.tight_layout()
plt.show()

# 7. VISUALIZACIÓN: Gráfico de Barras Comparativo
fig, ax = plt.subplots(figsize=(10, 6))

metrics = ['Accuracy', 'Recall', 'Precision']
con_fe = [

```

```

acc_completo,
recall_completo,
cm_completo[1,1] / (cm_completo[1,1] + cm_completo[0,1]) # Precision
]
sin_fe = [
acc_no_fe,
recall_no_fe,
cm_no_fe[1,1] / (cm_no_fe[1,1] + cm_no_fe[0,1]) # Precision
]

x = np.arange(len(metrics))
width = 0.35

bars1 = ax.bar(x - width/2, con_fe, width, label='CON Feature Engineering',
color='green', alpha=0.7)
bars2 = ax.bar(x + width/2, sin_fe, width, label='SIN Feature Engineering',
color='orange', alpha=0.7)

ax.set_xlabel('Métricas', fontsize=12)
ax.set_ylabel('Score', fontsize=12)
ax.set_title('Impacto del Feature Engineering en el Rendimiento del Modelo',
fontsize=14, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(metrics)
ax.legend()
ax.grid(axis='y', alpha=0.3)

# Añadir valores en las barras
for bars in [bars1, bars2]:
    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height,
f'{height:.3f}',
ha='center', va='bottom', fontsize=9, fontweight='bold')

plt.tight_layout()
plt.show()

```

Error:


```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 20, in <module>
NameError: name 'X_train' is not defined
```

Cell 81: ■ Markdown

Conclusiones: Validación del Feature Engineering

Tras entrenar el **Random Forest Optimizado** con y sin las variables de ingeniería, los resultados revelan una **conclusión sorprendente e importante** para la estrategia de modelado.

Análisis del Impacto

El Feature Engineering NO Aportó Valor Predictivo

Contra toda expectativa inicial, las **7 variables creadas manualmente** (`Tenure_Group`, `Num_Services`, `Has_Family`, `relative_monthly_charge`) no solo **no mejoraron el modelo**, sino que lo **empeoraron ligeramente**:

1. **Pérdida en Recall (-0.8%):**

- El modelo SIN las variables creadas detecta **3 clientes adicionales** que se van (280 vs 277 TP).
- Reduce los falsos negativos de 97 a 94, salvando a **3 clientes más**.

2. **Mejora en Accuracy (+0.35%):**

- El modelo con feature engineering es ligeramente más preciso globalmente

3. **Simplificación Significativa:**

- Pasar de **30 features a 23** (reducción del 23%) sin perder rendimiento.
- Menos complejidad = más robusto, más rápido, más fácil de mantener.

¿Por Qué Falló el Feature Engineering?

Existen varias explicaciones posibles:

1. Redundancia con Variables Originales

- `Tenure_Group` era una versión categorizada de `tenure`, pero Random Forest ya puede capturar la no-linealidad de `tenure` mediante splits múltiples.
- `Has_Family` combinaba `Partner` y `Dependents`, pero el modelo ya podía aprender esa interacción de forma implícita.

2. El Poder de Random Forest

- Los árboles de decisión son **extremadamente buenos** capturando interacciones no lineales entre variables.

- No necesitan que les "mastiquemos" las transformaciones: ellos mismos encuentran los puntos de corte óptimos.

- **Ejemplo:** En lugar de categorizar `tenure` en grupos predefinidos (0-12, 12-24, etc.), Random Forest puede decidir que el corte óptimo está en `tenure < 15 meses` para un nodo específico.

Con base en estos resultados, la decisión es clara:

Para Producción: Usar el Modelo SIN Feature Engineering

Ventajas:

1. **Mejor Rendimiento:** +3 clientes salvados (74.87% vs 75.7% recall).
2. **Menos Complejidad:** 23 features en lugar de 30 (-23% de dimensionalidad).
3. **Menor Riesgo de Overfitting:** Menos variables = modelo más generalizable.
4. **Mantenimiento Simplificado:** No hay que recalcular `Tenure\_Group`, `relative\_monthly\_charge`, etc. en cada predicción.
5. **Más Rápido:** Menos features = menor tiempo de inferencia en producción.

Desventajas:

- Ninguna relevante. El modelo simplificado es objetivamente superior.